

[ASP.NET](#) Core 8 — Entity Framework Core — PostgreSQL

DOCUMENTATION DE PROJET

Master



Professeur : Gregory SOLIDE

Niveau : MBDS / UEH / FDS

Auteurs : Jasmin MICHEL / Josselet ORELUS / Huguens AIME

Git : <https://github.com/MBDS-MICHEL-ORELUS-AIME/Micro-Plateforme-E-Learning.git>

version : 1.2

1. Introduction

1.1 Contexte du projet

Dans le cadre du cursus académique MBDS (Master Base de Donnée & Intégration Système), ce projet répond à la demande PDR 2 portant sur la conception et le développement d'une micro-plateforme d'e-learning. La plateforme est construite sur la stack **ASP.NET Core 8 MVC** avec une base de données **PostgreSQL**, suivant une architecture en couches (Core, Services, Web).

Le projet vise à proposer un environnement d'apprentissage en ligne simple, accessible et maintenable, sans la complexité d'un LMS (Learning Management System) de niveau entreprise. Il s'inscrit dans une démarche de valorisation des compétences **.NET** acquises en formation, tout en couvrant les besoins essentiels d'une plateforme éducative moderne.

1.2 Problématique

De nombreux établissements d'enseignement et centres de formation font face à un besoin croissant d'outils numériques pour organiser et diffuser leurs contenus pédagogiques. Les solutions LMS existantes (Moodle, Canvas, Udemy, Openclassroom, etc.) sont souvent trop complexes à déployer, trop coûteuses à maintenir, ou inadaptées à des contextes académiques à ressources limitées.

La problématique centrale du projet est donc la suivante :

Comment concevoir une plateforme e-learning fonctionnelle, économique à opérer et simple à maintenir, qui couvre les besoins essentiels d'un parcours apprenant (cours, quiz, progression, certification) tout en restant accessible à des équipes techniques de taille réduite ?

1.3 Objectif du projet

L'objectif général est de livrer une plateforme web opérationnelle permettant à des apprenants de suivre des cours structurés, d'être évalués via des quiz, de visualiser leur

progression et d'obtenir des certificats de complétion, le tout géré par des enseignants et des administrateurs via des interfaces dédiées.

1.3.1 Objectif pédagogiques (cours)

- ❖ Permettre la structuration du contenu en modules et leçons (texte, vidéo, PDF).
- ❖ Assurer le **suivi de progression** leçon par leçon, avec calcul du taux de complétion global par module.
- ❖ Proposer un moteur de quiz supportant plusieurs types de questions (QCM, vrai/faux, réponse courte) afin de valider les acquis des apprenants.
- ❖ Émettre des certificats de complétion au format PDF, vérifiables publiquement via un code unique, à l'issue des modules complétés.
- ❖ Offrir un espace de discussion structuré (fils de discussion, réponses, statut résolu/non résolu) pour favoriser les échanges entre apprenants et enseignants.

1.3.2 Objectifs fonctionnels (produit)

- ❖ Fournir une interface ****apprenant**** (tableau de bord, lecteur de cours, historique de quiz, badges).
- ❖ Fournir un espace de travail enseignant pour créer et gérer modules, leçons, quiz et médias.
- ❖ Fournir un tableau de bord administrateur (superadmin) centralisant la gestion des utilisateurs, la modération des contenus, la synchronisation de contenu externe et les indicateurs d'activité.
- ❖ Intégrer un pipeline d'import de contenu externe (sources ouvertes : Wikiversity, Wikipedia, Project Gutenberg) pour enrichir la plateforme sans budget.
- ❖ Garantir la sécurité des accès par un système d'authentification et d'autorisation basé sur les rôles (étudiant, enseignant, coordinateur, superadmin).

1.4 Périmètre du projet

Le périmètre couvre les domaines fonctionnels suivants :

Domaine → *Inclus dans le périmètre*

Gestion des utilisateurs → Inscription, connexion, gestion des rôles par l'admin

Catalogue de cours → Création, lecture, navigation par module et leçon

Médias pédagogiques → Texte, vidéo (URL externe), PDF (upload)

Évaluation → Quiz (QCM, vrai/faux, réponse courte), historique, score
Progression → Suivi par leçon, taux de complétion, badges
Certification → Génération PDF, vérification publique par code
Discussion → Fils de discussion, réponses, signalement, résolution
Administration → Tableau de bord, gestion utilisateurs, modération, synchronisation contenu
Import de contenu → Pipeline API (Wikiversity/Wikipedia), registre de traçabilité
Sont hors périmètre : la facturation/monétisation, les notifications push, les intégrations SCORM/xAPI, et le support mobile natif.

2. Analyse des besoins

2.1 Identification des acteurs

La plateforme distingue quatre profils d'utilisateurs, chacun disposant d'un niveau d'accès et d'un ensemble de fonctionnalités spécifiques :

Acteur → *Description. Accès principal*

Étudiant → Apprenant inscrit sur la plateforme. Rôle attribué automatiquement à l'inscription. Tableau de bord apprenant, cours, quiz, discussion, certificats

Enseignant → Formateur créant et gérant les contenus pédagogiques. Espace de travail enseignant, gestion modules/leçons/quiz/médias.

Coordinateur → Responsable pédagogique supervisant l'activité globale de la plateforme. Dashboard consolidé, vue statistiques, modération

Superadmin → Administrateur technique disposant de tous les droits. Gestion complète : utilisateurs, rôles, synchronisation contenu, modération.

2.2 User Stories Principales

2.2.1 En tant qu'Étudiant

- US-E01 : En tant qu'étudiant, je veux m'inscrire sur la plateforme avec un nom d'utilisateur, un e-mail et un mot de passe, afin d'accéder à mon espace personnel.
- US-E02 : En tant qu'étudiant, je veux consulter le catalogue des cours disponibles, afin de choisir un module.

-
- US-E03 : En tant qu'étudiant, je veux lire les leçons d'un module (texte, vidéo, PDF), afin de progresser dans mon parcours d'apprentissage.
 - US-E04 : En tant qu'étudiant, je veux marquer une leçon comme lue, afin que ma progression soit enregistrée.
 - US-E05 : En tant qu'étudiant, je veux passer un quiz associé à un module, afin d'évaluer mes connaissances acquises.
 - US-E06 : En tant qu'étudiant, je veux consulter mon historique de quiz (score, date, résultat), afin de suivre mon évolution.
 - US-E07 : En tant qu'étudiant, je veux télécharger mon certificat de complétion au format PDF, afin de valoriser ma formation.
 - US-E08 : En tant qu'étudiant, je veux consulter mes badges obtenus, afin de visualiser mes accomplissements.
 - US-E09 : En tant qu'étudiant, je veux créer un fil de discussion et y répondre, afin d'échanger avec les autres apprenants et les enseignants.

2.2.2 En tant qu'Enseignant

- US-T01 : En tant qu'enseignant, je veux créer un module de cours (titre, description), afin d'organiser mon contenu pédagogique.
- US-T02 : En tant qu'enseignant, je veux ajouter des leçons à un module (texte, URL vidéo, PDF), afin de structurer le contenu de manière progressive.
- US-T03 : En tant qu'enseignant, je veux uploader des médias (PDF, images) liés à mes leçons, afin d'enrichir le contenu disponible.
- US-T04 : En tant qu'enseignant, je veux créer et modifier des quiz (questions QCM, vrai/faux, réponse courte) associés à mes modules, afin d'évaluer les apprenants.
- US-T05 : En tant qu'enseignant, je veux supprimer un quiz, afin de gérer le cycle de vie de mes évaluations.
- US-T06 : En tant qu'enseignant, je veux consulter la liste de mes quiz, afin d'avoir une vue d'ensemble de mes contenus d'évaluation.

2.2.3 En tant que Coordinateur

- US-C01 : En tant que coordinateur, je veux accéder à un tableau de bord consolidé, afin de superviser l'activité globale de la plateforme (inscriptions, progressions, résultats de quiz).

-
- US-C02 : En tant que coordinateur, je veux consulter les indicateurs de performance clés (taux de complétion, taux de réussite aux quiz, certificats émis), afin d'évaluer la qualité des parcours.
 - US-C03 : En tant que coordinateur, je veux modérer les discussions signalées, afin de maintenir un environnement d'apprentissage sain.

2.2.4 En tant qu'Administrateur

- US-A01 : En tant qu'administrateur, je veux consulter un tableau de bord avec les indicateurs d'activité (utilisateurs actifs, inscriptions, progressions sur une période donnée), afin de piloter la plateforme.
- US-A02 : En tant qu'administrateur, je veux gérer les comptes utilisateurs (créer, modifier, supprimer, changer de rôle), afin de contrôler les accès à la plateforme.
- US-A03 : En tant qu'administrateur, je veux déclencher et suivre la synchronisation de contenus externes (Wikiversity, Wikipedia), afin d'enrichir le catalogue de cours.
- US-A04 : En tant qu'administrateur, je veux gérer les sources PDF des leçons et actualiser leur référencement, afin d'assurer la disponibilité des ressources.
- US-A05 : En tant qu'administrateur, je veux modérer les signalements de discussions (approuver, rejeter), afin de maintenir la qualité des échanges.
- US-A06 : En tant qu'administrateur, je veux vérifier un certificat via son code unique, afin de garantir l'authenticité des attestations émises.

2.3 Exigences fonctionnels

EF-01 — Authentification et gestion des accès

- La plateforme doit permettre l'inscription avec attribution automatique du rôle « étudiant ».
- La connexion doit être sécurisée (hachage des mots de passe, jeton anti-falsification CSRF).
- L'accès à chaque section doit être contrôlé par le rôle de l'utilisateur connecté.

EF-02 — Gestion des cours

- Un module regroupe plusieurs leçons ordonnées.
- Une leçon peut contenir du texte, une URL vidéo externe et/ou un fichier PDF.

-
- Le catalogue de cours doit être consultable avec recherche textuelle, filtre par quiz disponible et pagination.

EF-03 — Progression apprenant

- La progression est calculée par module, sur la base des leçons marquées comme lues.
- Le tableau de bord apprenant affiche le taux de complétion global et par module.

EF-04 — Quiz et évaluation

- Un quiz est composé de questions de type QCM, vrai/faux ou réponse courte.
- Le résultat d'un quiz (score, date) est persisté et consultable dans l'historique de l'apprenant.
- Un apprenant peut repasser un quiz.

EF-05 — Certification

- Un certificat est généré automatiquement lorsqu'un apprenant complète 100 % d'un module.
- Chaque certificat est identifié par un code unique permettant une vérification publique sans authentification.

EF-06 — Discussion

- Tout utilisateur connecté peut créer un fil de discussion et y répondre.
- L'auteur d'un fil ou un enseignant peut le marquer comme résolu.
- Un utilisateur peut signaler un fil inapproprié ; les signalements sont traités par un modérateur ou l'administrateur.

EF-07 — Import de contenu externe

- L'administrateur peut déclencher une synchronisation depuis des sources ouvertes (Wikiversity, Wikipedia, Project Gutenberg).
- Chaque contenu importé doit conserver ses métadonnées de traçabilité (source, URL, licence, date d'import).
- Un tableau de bord de synchronisation affiche les volumes, les erreurs et la date de la dernière synchronisation.

- L'administrateur dispose d'un tableau de bord avec indicateurs filtrables par période.
- La gestion des utilisateurs couvre la création, la modification du rôle et la suppression de comptes.
- La modération des discussions signalées est accessible depuis l'interface d'administration.

3. Architecture du Système

3.1 Stack Technologiques

Couche	Technologie	Version	Rôle
Présentation	ASP.NET Core MVC + Razor Views	.NET 8	Interface web côté serveur
Frontend UI	Bootstrap + jQuery + HTMX + JavaScript	Mixte	Responsive UI et interactions dynamiques
API / Routage	ASP.NET Core Controllers (HTTP)	.NET 8	Endpoints GET/POST pour fonctionnalités métier
Métier	C# (Domain + Services applicatifs)	Version: C# 12	Règles métier, progression, quiz, certificats
Persistence ORM	Entity Framework Core + Design	8.0.19	Mapping objet-relationnel, migrations
Provider SQL	Npgsql.EntityFrameworkCore.PostgreSQL	8.0.8	Connexion EF Core vers PostgreSQL
Base de données	PostgreSQL	14+	Stockage relationnel principal
Authentification	Session ASP.NET Core + rôle applicatif	.NET 8	Gestion login/logout et contrôle d'accès

3.2 Architecture en couche N-Tier

L'application suit une architecture en couche N-Tier strictement séparées, conformément aux principes SOLID, avec séparation Présentation, API, Application, Domaine et Infrastructure, plus une couche de tests transverse.

- Couche Présentation (MVC) : Controllers Razor, Views .cshtml, ViewModels, interactions utilisateur

- Couche API : Endpoints REST retournant du JSON pour l'intégration front/externe
- Couche Application : Services applicatifs, orchestration des cas d'usage, DTO/mapping
- Couche Domaine : Entités métier, règles métier, contrats (interfaces)
- Couche Infrastructure : Accès aux données (EF Core), implémentation des repositories, services techniques externes
- Couche Tests (transverse) : Tests d'intégration et validation des flux fonctionnels

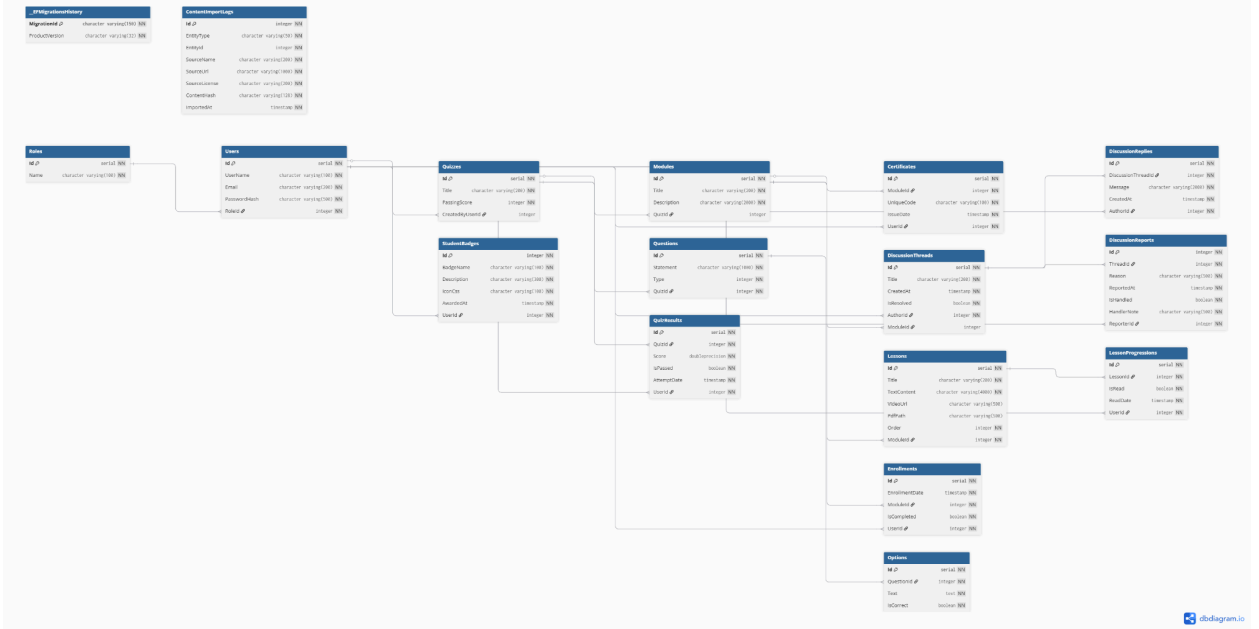
3.3 Modèle de données (Entités Principales)

Package : E-learningProject.Core		
Entité	Attribut clés	Relation
User	Id, UserName, Email, PasswordHash, RoleId	n..1 Role
Role	Id, Name	1..n Users
Module	Id, Title, Description, QuizId (nullable)	1..n Lessons, 0..1 Quiz
Lesson	Id, Title, TextContent, VideoUrl, PdfPath, Order, ModuleId	n..1 Module, 1..n LessonProgressions
Quiz	Id, Title, PassingScore	1..n Questions, 1..n QuizResults, 0..n Modules
Question	Id, Statement, Type, QuizId	n..1 Quiz, 1..n Options
Option	Id, QuestionId, Text, IsCorrect	n..1 Question
Enrollment	Id, EnrollmentDate, StudentId, ModuleId, IsCompleted	n..1 Module
LessonProgression	Id, StudentId, LessonId, IsRead, ReadDate	n..1 Lesson
QuizResult	Id, StudentId, QuizId, Score, IsPassed, AttemptDate	n..1 Quiz
Certificate	Id, StudentId, ModuleId, UniqueCode, IssueDate	n..1 Module

Package : E-learningProject.Core

Entité	Attribut clés	Relation
DiscussionThread	Id, Title, StudentId, CreatedAt, IsResolved	1..n DiscussionReplies
DiscussionReply	Id, DiscussionThreadId, AuthorId, Message, CreatedAt	n..1 DiscussionThread
ContentImportLog	Id, EntityType, EntityId, SourceName, SourceUrl, SourceLicense, ContentHash, ImportedAt	Journal technique (pas de FK explicite)

3.4 Schéma de base de données



3.5 Sécurité et Authentification

Insérez votre texte ici Insérez votre texte ici Insérez votre texte ici Insérez votre texte ici
Insérez votre texte ici Insérez votre texte ici Insérez votre texte ici.

4. Fonctionnalités Développées

4.1 Module Authentication

Le module d'authentification est implémenté dans `AccountController` et repose sur une gestion de session côté serveur (ASP.NET Core Session) combinée à un système de hachage de mots de passe personnalisé (`PasswordSecurity`).

Fonctionnalités implémentées :

- **Inscription** : Un visiteur peut créer un compte en fournissant un nom d'utilisateur, une adresse e-mail et un mot de passe. Le système vérifie l'unicité du nom d'utilisateur et de l'e-mail avant création. Le rôle `etudiant` est attribué automatiquement. Le mot de passe est haché avant persistance.
- **Connexion** : Authentification par nom d'utilisateur et mot de passe. Le système compare le mot de passe fourni au hash stocké. En cas de succès, les informations de session (`CurrentUserId`, `CurrentUserName`, `CurrentUserRole`) sont écrites. Un mécanisme de migration silencieuse convertit les mots de passe en clair hérités vers un hash sécurisé lors de la première connexion réussie.
- Redirection par rôle Après connexion ou inscription, l'utilisateur est redirigé automatiquement vers l'interface correspondant à son rôle (apprenant, enseignant, coordinateur, superadmin).
- **Déconnexion** : La session est entièrement invalidée (suppression des clés de session).
- Protection CSRF : Tous les formulaires POST utilisent le jeton anti-falsification `ValidateAntiForgeryToken`.

Sécurité :

- Les mots de passe ne sont jamais stockés en clair.
- La redirection après connexion est validée comme URL locale (`Url.IsLocalUrl`) pour prévenir les attaques de redirection ouverte.
- L'accès aux sections protégées est conditionné à la présence de la session active.

4.2 Module Catalogue de cours

Le catalogue est géré par `CoursesController` (action `Index`) et offre aux utilisateurs une vue d'ensemble de tous les modules disponibles sur la plateforme.

Fonctionnalités implémentées :

- Affichage paginé : Les modules sont affichés par page de 6 (configurable, max 24). La pagination est gérée côté serveur avec `Skip`/`Take`.
- Recherche textuelle : L'utilisateur peut filtrer les modules par mot-clé sur le titre et la description (recherche insensible à la casse).
- Filtre par quiz : Trois modes de filtre — tous les modules (`all`), uniquement ceux avec un quiz final (`withquiz`), uniquement ceux sans quiz (`withoutquiz`).
- Indicateurs globaux : Le bandeau supérieur affiche le nombre total de modules, leçons, quiz disponibles, inscriptions et completions globales.
- Détail d'un module : La page de détail affiche la liste des leçons, le taux de complétion de l'apprenant connecté (basé sur

4.3 Module Lecture de Cours

La lecture des cours est gérée par `LearnerController` (action `Reader`) et constitue le cœur de l'expérience apprenant.

Fonctionnalités implémentées :

- Lecteur de leçon : Affichage du contenu textuel de la leçon courante, avec rendu de la vidéo (intégrée ou URL externe) et du document PDF associé si disponibles.
- Navigation entre leçons : Les leçons sont ordonnées (`Order`, puis `Id`) et l'utilisateur peut naviguer séquentiellement.

-
- Marquage de leçon lue : L'action `MarkRead` persiste la progression en base (`LessonProgression`). Une leçon ne peut être marquée qu'une seule fois (idempotent).
 - Tableau de bord apprenant : La vue `Dashboard` calcule en temps réel, pour chaque module, le nombre de leçons lues vs total et le pourcentage de complétion via `IProgressService`. Elle agrège également : modules complétés, certificats obtenus, tentatives et réussites de quiz, discussions ouvertes.
 - Badges : Les 4 derniers badges obtenus par l'apprenant sont affichés sur le tableau de bord (nom, description, icône CSS, date d'attribution).
 - Historique de quiz : L'action `QuizHistory` liste tous les résultats de quiz de l'apprenant (score, date, réussi/échoué).

4.4 Module Quiz et Évaluation

Le moteur de quiz est implémenté dans `QuizController` avec le service métier `IQuizService`. Il supporte un passage question par question avec persistance des réponses intermédiaires en session.

Fonctionnalités implémentées :

- Types de questions : QCM (choix unique parmi les options), vrai/faux (cas particulier de QCM), réponse courte (texte libre comparé à une réponse de référence).
- Passage question par question : L'action `Question` renvoie une vue partielle (`\_QuizQuestionStep`) pour chaque question. La progression (index, pourcentage) est affichée en temps réel. Les réponses intermédiaires sont stockées en session JSON et restaurées si l'utilisateur navigue en arrière.
- Correction automatique : À la soumission finale (`Finish`), chaque réponse est comparée à la bonne réponse (option correcte pour QCM, comparaison insensible à la casse pour la réponse courte). Un récapitulatif de correction est généré avec la réponse donnée, la bonne réponse et le statut.

-
- Score et seuil de réussite : Le score est calculé en pourcentage de bonnes réponses et comparé au `PassingScore` du quiz. Le résultat (`QuizResult`) est persisté en base avec date, score et statut.
 - Page de résultats : La vue `Result` affiche le score, le statut (réussi/échoué), la correction détaillée question par question et un lien vers le téléchargement du certificat si éligible.
 - Catalogue de quiz : L'action `Index` liste tous les quiz disponibles avec leur nombre de questions et module associé.

4.5 Module Enseignant

L'espace enseignant est entièrement géré par `TeacherController`. L'accès est restreint aux utilisateurs ayant le rôle `enseignant` ou `superadmin` (garde `CanTeach()`).

Fonctionnalités implémentées :

- Espace de travail (`Workspace`) : Interface unifiée permettant de gérer modules, leçons et médias depuis un seul écran. Le module sélectionné est mémorisé via paramètre d'URL.
- Création de module : Formulaire de création d'un module (titre, description) avec validation serveur.
- Création de leçon : Ajout d'une leçon à un module existant avec contenu texte, URL vidéo externe (validée http/https) et/ou fichier PDF uploadé. Les fichiers sont stockés dans `wwwroot/uploads/`. L'ordre d'affichage est paramétrable.
- Validation des uploads : Le PDF est limité à 10 Mo, les vidéos à 100 Mo. Les extensions autorisées sont contrôlées côté serveur (`.pdf` / `.mp4`, `.webm`, `.mov`, `.avi`, `.mkv`). Les fichiers non conformes sont rejetés avec un message d'erreur explicite.
- Gestion des quiz : Création, modification et suppression de quiz. Un quiz est composé d'un titre, d'un seuil de réussite et d'une liste de questions (type, énoncé, options, bonne réponse). L'enseignant peut associer un quiz à un module.
- Vue Mes Quiz (`MyQuizzes`) : Tableau récapitulatif de tous les quiz avec statistiques agrégées : nombre total de tentatives, tentatives réussies, score moyen.

-
- Upload de médias : Action dédiée `UploadMedia` permettant d'associer des fichiers supplémentaires à des leçons existantes.

4.6 Module Administrateur

Le module administrateur est géré par `AdminController`. L'accès au dashboard est ouvert aux rôles `superadmin` et `coordinateur` ; la gestion des utilisateurs et la synchronisation de contenu sont réservées au `superadmin`.

Fonctionnalités implémentées :

- Tableau de bord (`Dashboard`) : Agrège en temps réel les indicateurs clés sur une période sélectionnable (7, 30 ou 90 jours) : total modules/leçons/quiz, inscriptions, completions, certificats émis, tentatives de quiz et taux de réussite. Les données incluent : top modules par inscriptions, taux de réussite par quiz, 5 derniers fils de discussion, 8 derniers certificats émis, répartition des utilisateurs par rôle.
- Gestion des utilisateurs : Création, modification (nom, email, rôle, mot de passe) et suppression de comptes. Les mots de passe administrateur sont hachés à la sauvegarde. Liste complète des utilisateurs avec leur rôle.
- Synchronisation de contenu externe (`ContentSync`) : Interface permettant de déclencher l'import de contenu depuis des API ouvertes (Wikiversity, Wikipedia) via `IOpenContentImportService`. Le paramètre `maxModules` est configurable. Un journal d'import (`ContentImportLog`) trace les résultats.
- Sources PDF des leçons (`LessonPdfSources`) : Vue listant les sources PDF référencées dans les leçons, avec action de rafraîchissement.
- Modération des discussions (`Moderation`) : Gestion des signalements (`DiscussionReport`) avec filtre par statut (en attente, traités). L'administrateur peut approuver ou rejeter un signalement via l'action `HandleReport`.
- Vérification de certificats : Accessible sans authentification, la page `Verify` permet de contrôler l'authenticité d'un certificat à partir de son code unique.

5. API REST - Endpoints principaux

Suivi des révisions			
Méthode	Endpoint	Rôle requis	Description
POST	/Account/Login	Public	Authentification utilisateur
GET	/Account/Login	Public	Formulaire de connexion
GET	/Teacher/Workspace	Enseignant, Coordinateur	Espace enseignant
POST	/Teacher/CreateModule	Enseignant, Coordinateur	Créer un module
GET	/Learner/Dashboard	Etudiant	Tableau de bord apprenant
POST	/Learner/MarkRead	Etudiant	Marquer une leçon lue (vue apprenant)

6. Stratégie de Tests

7. Déploiement & Infrastructure

7.1 Architecture de la solution

La solution est organisée en **4** projets **.NET** avec une séparation claire des responsabilités :

Projet —> Framework —> Rôle

E-learningProject.Core — .NET — Entités du domaine, énumérations

E-learningProject.Data — .NET — ApplicationDbContext (EF Core), repositories

E-learningProject.Services — .NET — Services métier (quiz, progression, certificats, import)

E-learningProject.Web — ASP.NET Core 9 — Contrôleurs MVC, vues Razor, Program.cs

Les dépendances de projet suivent la direction `Web → Services → Data → Core`, sans couplage inverse.

7.2 Pré Requis techniques

Composant → Version minimale

.NET SDK 9.0

PostgreSQL 14+

PowerShell | 5.1+ (Windows)

7.3 Configuration de la base de données

La connexion PostgreSQL est résolue dans `Program.cs` selon l'ordre de priorité suivant :

1. Variable d'environnement **`MICROLMS\_CONNECTION\_STRING`** (priorité haute)* — chaîne de connexion complète, surcharge tout.

2. `appsettings.json` → `ConnectionStrings:DefaultConnection` (priorité basse) — la valeur par défaut contient le marqueur ``CHANGE_ME`` pour éviter tout commit de credentials réels.

Paramètres par défaut (local) :

Paramètre → Valeur

Hôte → localhost

Port → 5432

Base de données → ``MicroLmsDb``

Utilisateur → ``postgres``

Search Path → ``public, lms``

Variables d'environnement supportées

Variable — Rôle — Priorité

``MICROLMS_CONNECTION_STRING`` → Chaîne de connexion complète : 1 (prioritaire)

``POSTGRES_PASSWORD`` → Mot de passe PostgreSQL (user ``postgres``, db ``MicroLmsDb``) 2

Règle de sécurité : Ne jamais committer de credentials réels dans `appsettings.json`. La valeur ``CHANGE_ME`` est un marqueur volontaire qui doit rester dans le dépôt.

7.4 Script de démarrage

Le fichier **`start-web.ps1`** centralise le démarrage de l'application en local :

powershell

```
$env:DOTNET_ROLL_FORWARD = 'Major'
```

```
$env:ASPNETCORE_ENVIRONMENT = 'Development'
```

```
$env:ASPNETCORE_URLS = 'http://localhost:5230'
```

Le script :

1. Arrête toute instance `E-learningProject.Web` déjà en cours.
 2. Construit la variable `MICROLMS\_CONNECTION\_STRING` à partir de `POSTGRES\_PASSWORD` si elle n'est pas déjà définie (mot de passe par défaut : `postgres`).
 3. Lance `dotnet run` sur le projet web sans profil de lancement (`--no-launch-profile`).
- L'application est accessible sur <http://localhost:5230> après démarrage.

7.5 Initialisation automatique au démarrage

Le comportement au démarrage est piloté par quatre drapeaux dans `appsettings.json` :

json

```
"Database": {  
  
  "ApplyMigrationsOnStartup": true,  
  
  "AutoSeedAcademicDataOnStartup": true,  
  
  "ImportOpenContentOnStartup": false,  
  
  "SeedDemoDataOnStartup": false  
}
```

Drapeau	Valeur par défaut	Comportement
---------	-------------------	--------------

---	---	---
-----	-----	-----

`ApplyMigrationsOnStartup`	`true`	Applique les migrations EF Core en attente. Si la base n'existe pas encore, `EnsureCreated` est appelé en amont.
----------------------------	--------	--

`AutoSeedAcademicDataOnStartup`	`true`	Injecte les modules, leçons et quiz académiques de référence si la base est vide. Crée également les 4 rôles requis.
---------------------------------	--------	--

| ``ImportOpenContentOnStartup`` | ``false`` | Déclenche le pipeline d'import de contenu externe (Wikiversity/Wikipedia) au démarrage. Désactivé par défaut. |

| ``SeedDemoDataOnStartup`` | ``false`` | Crée les comptes de démonstration et les données d'exemple. Désactivé par défaut. |

7.6 Comptes et rôles créés automatiquement

Les 4 rôles sont créés automatiquement à la première initialisation :

Rôle	Description
------	-------------

---	---
-----	-----

<code>`etudiant`</code>	Apprenant — rôle attribué à l'inscription publique
-------------------------	--

<code>`enseignant`</code>	Formateur — accès à l'espace de travail enseignant
---------------------------	--

<code>`coordinateur`</code>	Responsable pédagogique — accès au dashboard consolidé
-----------------------------	--

<code>`superadmin`</code>	Administrateur — accès complet à toutes les fonctionnalités
---------------------------	---

Lorsque ``SeedDemoDataOnStartup`` est activé (``true``), les comptes de démonstration suivants sont créés (si absents) :

Nom d'utilisateur	Email	Rôle	Mot de passe
-------------------	-------	------	--------------

---	---	---	---
-----	-----	-----	-----

<code>`superadmin`</code>	<code>`superadmin@microlms.local`</code>	superadmin	<code>`Admin@123`</code>
---------------------------	--	------------	--------------------------

```
| `coordinateur.demo` | `coordinateur@microlms.local` | coordinateur | `Admin@123` |  
| `teacher.demo` | `teacher@microlms.local` | enseignant | `Admin@123` |  
| `student.demo` | `student@microlms.local` | etudiant | `Admin@123` |
```

> Ces comptes sont destinés exclusivement aux environnements de développement et de démonstration. Ils ne doivent pas être activés en production.

7.7 Gestion de la session

La session utilisateur est configurée dans `Program.cs` avec les paramètres suivants :

- **Durée d'inactivité** : 30 minutes (`IdleTimeout`).
- **Cookie `HttpOnly`** : activé (inaccessible depuis JavaScript).
- **Cookie `IsEssential`** : activé (pas soumis au consentement RGPD côté serveur).
- **Stockage** : `DistributedMemoryCache` (en mémoire locale, adapté à un déploiement single-instance).

Les clés de session utilisées sont : `CurrentUserId`, `CurrentUserName`, `CurrentUserRole`.

7.8 Procédure de déploiement local (première fois)

```
```powershell
```

---

# 1. Déclarer le mot de passe PostgreSQL pour la session

\$env:POSTGRES\_PASSWORD = "votre\_mot\_de\_passe"

# 2. Lancer l'application (migrations + seeding automatiques)

.\start-web.ps1

...

L'application sera disponible sur `http://localhost:5230`. La structure de base, les rôles et les données académiques sont créés automatiquement au premier démarrage — **aucune** commande SQL manuelle n'est nécessaire.

### 7.9 Considérations pour un déploiement en production

| Point | Recommandation |

|---|---|

| Chaîne de connexion | Utiliser `MICROLMS\_CONNECTION\_STRING` via secret manager ou variable d'environnement système, jamais en dur dans les fichiers de config |

| Session | Remplacer `DistributedMemoryCache` par Redis ou SQL Server si déploiement multi-instance |

| Fichiers uploadés | Externaliser `wwwroot/uploads/` vers un stockage persistant (Azure Blob, S3, NFS) |

| HTTPS | Configurer un reverse proxy (nginx, Caddy) avec certificat TLS |

| `SeedDemoDataOnStartup` | Laisser à `false` — ne jamais activer en production |

---

| `ImportOpenContentOnStartup` | Désactiver (`false`) et préférer un déclenchement manuel via le dashboard admin |

## 8. Difficultés rencontrées et Solutions

## 9. Bilan du projet & Perspective

## 10. Glossaire

US : User Story (*US-E01n : User Story Etudiant n / US-T01n : User Story Teacher n / US-C01n : User Story Coordinator n*)

EF : Exigence Fonctionnel (*EF-0n : Exigence fonctionnelle n*)